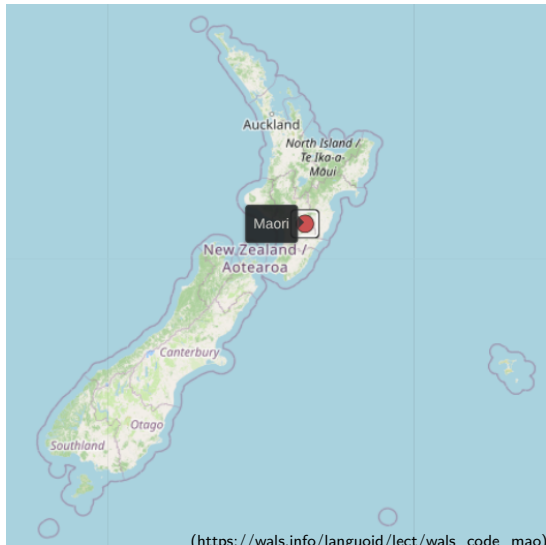


Māori via Xfst

Computational Morphology Summer 2026

Marie Vangerow-Kühn



(https://wals.info/languoid/lect/wals_code_mao)

Files

github.com/marievk/maori-xfst/

- |
- ├── README.md short intro
- ├── data/... test data
- ├── docs/... documentation
- └── src/
 - ├── blocks/...
 - ├── maori.xfst
 - ├── maori-guesser.xfst
 - └── maori-lexc.xfst

Main Features

Focus: verbal morphology (incl. universals and statives)

Key Aspects:

- tagging for different word forms
- nominalization: verbal nouns, agentive, reduplication
- passive (-a, ia, ...)
- validity: restricted alphabet and (C)V(V) as syllable structure

Basic Branching (simplified) - lexc-file

LEXICON Root

Bases ;
Nouns ;
Verbs ;
Statives ;
Locatives ;
Personals ;
Particles ;
Prefixes ;

LEXICON Bases ! universals
aru NandV ; ! V: "to follow", N: "pursuit"

LEXICON NandV

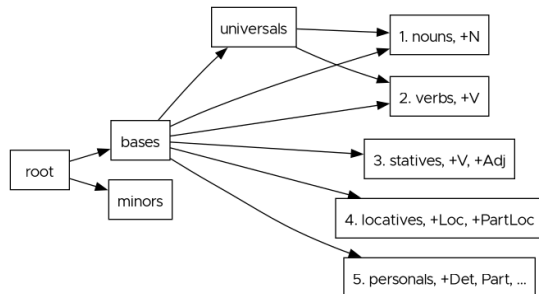
NTag ;
VTag ;

LEXICON NTag

+N:0 # ;

LEXICON VTag

+V:0 # ;



Nominalization: Verbal Nouns (-nga) - continuation classes

LEXICON Verb

noho VNga ; ! V: "to sit, stay" => nohoanga "place for sitting, chair"

kaa VNga ; ! V: "to take fire, burn" => kaainga "home, place where fire is lighted"

tika VNga ; ! V: "to be correct, upright" => tikanga N: "method, rule"

LEXICON VNga

VTag ; ! verb

+nom:nga NTag ; ! noun

LEXICON VNga

VTag ;

+nom:nga NTag ;

LEXICON VNga

VTag ;

+nom:nga NTag ;

⇒ lower: noho, nohoanga, upper: noho+V, noho+nom+N

(also implemented for -hanga, -ranga, -kanga, -tanga and -manga)

Nominalization: *kai*-Prefix (agentive) - Flag Diacritics

Multichar_Symbols +V +N +ag @P.PREF.Kai@ @D.PREF.Kai@ @R.PREF.Kai@

LEXICON root

@P.PREF.Kai@ag+:@P.PREF.Kai@kai KaiVerbs ;
KaiVerbs ;

LEXICON KaiVerbs

tuhi KaiAgentiv ; ! V: "to write" => kaituhi "writer, author"

LEXICON KaiAgentiv

@R.PREF.Kai@ NTag ; ! R: require agentive prefix

@D.PREF.Kai@ VTag ; ! D: disallow prefix

LEXICON NTag

+N:0 # ;

LEXICON VTag

+V:0 # ;

⇒ lower: kaituhi, tuhi, upper: ag+tuhi+N, tuhi+V

Nominalization: Full-Stem Reduplication - RegEx in lexc-file

```
Multichar_Symbols ^[ ^] R+ +V +N
```

```
LEXICON Root
```

```
0:^[{ VerbDupNomin ; ! V => N
```

```
LEXICON VerbDupNomin
```

```
meke VDupNBrack ; ! meke "to punch" => mekemeke "boxing"
```

```
LEXICON VDupNBrack
```

```
+V:}^] # ; ! ordinary form (verb)
```

```
^R+N:}^2^] # ; ! reduplicated form: verb => noun
```

⇒ lower: `^[{meke}^]`, `^[{meke}^2^]`, upper: `meke+V`, `meke^R+N`

compile-replace lower; ⇒ lower: meke, mekemeke

Passive - continuation classes

LEXICON Verb

```
homai VTag ; ! "give, contribute, grant, provide"
homai hiaPassive ; ! -> homaihia
homai tiaPassive ; ! -> homaitia
homai ØPassive ; ! -> homai
```

LEXICON tiaPassive
+pass:tia VTag ;

LEXICON hiaPassive
+pass:hia VTag ;

LEXICON ØPassive
+pass:Ø VTag ;

(also implemented for -a, -na, -nga, -ia, -tia, -hia, -ngia, -kia, -mia, -ria, -ina, -whia, -whina, -hina, -kina)

Initial partial reduplication - context-sensitive rewrite rules

Multichar_Symbols V+ CV^R+ Con Vow

```
LEXICON IniVDup
CV^R+:ConVow CloseIniVDup ;
          CloseIniVDup ;
```

```
LEXICON CloseIniVDup
patu VTag ; ! "to strike, hit" -> papatu "beat one another"
taki VTag ; ! "to lead, challenge" -> tataki "to lead/guide in succession"
```

⇒ upper-words: CV^R+taki+V, taki+V, lower-words: ConVowtaki, taki

```
define CopyConInit [Con -> p || _ Vow p,, Con -> t || _ Vow t] ;
define CopyVowInit [Vow -> a || _ ? a ,, Vow -> u || _ ? u] ;
read regex CopyConInit .o. CopyVowInit; ! sample excerpt, applied to all letters
```

⇒ lower-words: tataki, taki (expresses repetition and intensifies a verb)

Validity

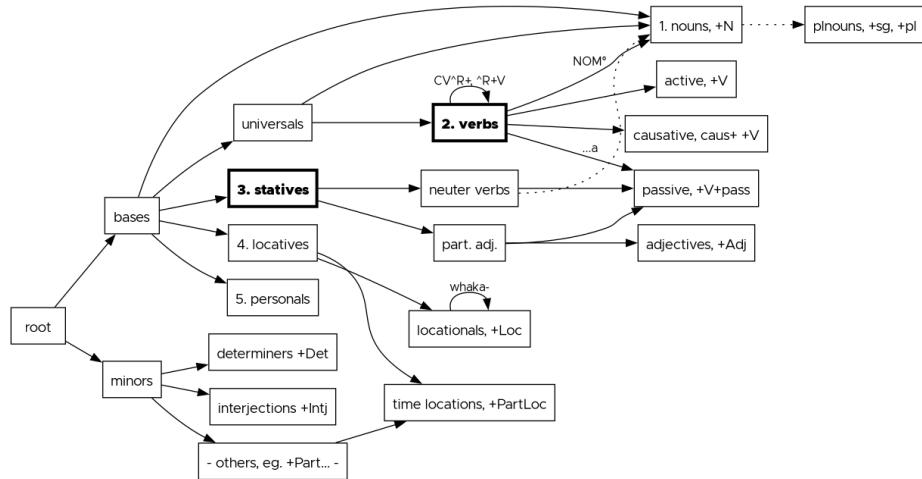
Restrict alphabet & syllable structure V, VV, CV, $\Rightarrow$ (C)V(V):

```
define C [h | k | m | n | ng | p | r | t | w | wh] ;
define V [a | e | i | o | u | ā | ē | ī | ō | ū] ;

define Syllable (C) V (V) ;
define Word Syllable+ ;
```

apply up chollöff ; $\Rightarrow$???

Lexicon Structure (simplified)



° Nominalization: ag+, ^R, -...nga

Core Operations / FST algebra

->	rewrite operation
	union, ' $[a b]^2$ ' => aa, bb, ab, ba
	context restriction
—	position of the rewritten material
*	Kleene star, zero or more repetitions
+	repeat once or more
?	optionality, '?*' => universal language
~	complement, '[?*' or '[a - a]', ... => empty language (0 arcs, 0 paths)
&	intersection, restrict to shared language
-	difference, remove unwanted strings
,	separation one one side of a rule
„	separates the whole rule
.o.	composition, connect two transducers
.x.	cross-product, map language to another

Flag Diacritics

Useful for separated or distanced constraints, syntax: `@type.feature.value@`, types:

P	positive	assigns feature value (just postulates)
N	negative	assigns complement of the value (rarely useful)
R	require	checks that feature-value pair is set - also: <code>@R.feature@</code>
D	disallow	checks that feature-value pair does not match (opposite of require)
C	clear	removes the value for the feature - also <code>@C.feature@</code>
U	unification	checks whether feature-value is compatible with previous setting & updates